

POLYMORPHISM

The word 'Polymorphism' is a Greek word, which means poly(many) and morph(forms).

Polymorphism in Java:

The ability of an object, that is class member to exhibit itself in more than one form is called Polymorphism.

```
1 package sampledemo;
2
3 class Demo {
4     public static void main(String[] args) {
5         int a = 10;
6         System.out.println(a);
7         a = 12;
8         System.out.println(a);
9         a = 24;
10        System.out.println(a);
11    }
12 }
13
```

<terminated> Demo [Java Ap
10
12
24

Here, variable 'a' is declared. At first it is initialized with value 10. Then, it is re-initialized with values 12 and 24. We can observe that the same variable container is able to hold more than one value at different times.

```
1 package sampledemo;
2
3 class Demo {
4     public static void abc() {
5         System.out.println("abc method");
6     }
7     public static void abc(int a) {
8         System.out.println("value passed: " + a);
9     }
10    public static void main(String[] args) {
11        abc();
12        abc(12);
13    }
14 }
15
```

<terminated> Demo [Java Ap
abc method
value passed: 12

Here, the method with name 'abc' is able to perform different actions based on the difference in the parameters.

These are basic examples of polymorphism we have come across so far.

TYPES OF POLYMORPHISM:

There are two types of Polymorphism in Java as follows:

1. Compile-Time Polymorphism
2. Run-Time Polymorphism

Compile Time Polymorphism:

The process of binding happens during compile-time and **same behaviour** is achieved in the run-time for Compile-time polymorphism. It is also called as Static polymorphism since same behaviour is achieved.

Compile-time polymorphism is the only area in OOPS concepts of Java, that allows static members also.

Techniques to achieve Compile time polymorphism:

1. Method Overloading
2. Constructor Overloading
3. Method Shadowing
4. Variable Shadowing
5. Operator Overloading

Operator Overloading:

It is not used in Java except for '+' operator, that performs both addition and concatenation based on the operands.

Method Shadowing:

If both parent and child classes have same methods with same name and same signature, it is called Method Shadowing.

Variable Shadowing:

If both parent and child classes have same variable with same name and same datatype, it is called Variable Shadowing.

Method Overloading:

If a class has more than one method with same name but different signatures, it is called Method Overloading.

Constructor Overloading:

If a class has more than one constructor with different signatures, it is called Constructor Overloading. Constructors have same name always.

Example for Method Shadowing and Variable Shadowing:

<pre>1 package sampledemo; 2 3 class ParentE 4 { 5 static int a = 12; 6 public static void abc() 7 { 8 System.out.println("parent abc method"); 9 } 10 } 11 12 class Demo extends ParentE 13 { 14 static int a = 24; 15 public static void abc() 16 { 17 System.out.println("child abc method"); 18 } 19 public static void main(String[] args) 20 { 21 System.out.println(a); 22 System.out.println(ParentE.a); 23 abc(); 24 ParentE.abc(); 25 } 26 } 27</pre>	<pre><terminated> Demo [Java App 24 12 child abc method parent abc method</pre>
---	---

Run Time Polymorphism:

The process of binding happens during compile-time and **different behaviour** is achieved in run-time for Run-time polymorphism. It is also called as Dynamic polymorphism since different behaviour is achieved.

Technique to achieve Run time polymorphism:

Method Over-riding

Method Over-Riding:

If both parent and child classes have same **non-static** methods with same name and same signature, it is called Method Over-riding.

Advantage of Method Over-riding:

1. New implementation is given to the parent class non-static method in the child class using a non-static method of same name and same signature.
2. The child class member (non-static method) is accessed with a object reference of Parent type.

Example for Method Over Riding:

```
1 package sampledemo;
2
3 class ParentE
4 {
5     int a = 12;
6     public void abc()
7     {
8         System.out.println("parent abc method");
9     }
10 }
11
12 class Demo extends ParentE
13 {
14     int a = 24;
15     public void abc()
16     {
17         System.out.println("child abc method");
18     }
19     public static void main(String[] args)
20     {
21         ParentE p = new Demo();
22         System.out.println(p.a);
23         // non static variable of parent is accessed
24         p.abc();
25         // we expect parent non static method to be accessed
26         // but non static method of child is accessed
27         // this is called different behavior
28     }
29 }
30
```

<terminated> Demo [Java Ap
12
child abc method